

HelixTranslate

HelixTranslate

Traduction de livres par modèle vérifié — conçue pour l'honnêteté, jamais un repli silencieux.

Résumé

HelixTranslate est une plateforme de traduction d'ebooks haute performance, basée sur Go, capable de traduire des livres entre plus de 100 langues en s'appuyant sur des fournisseurs LLM vérifiés. Elle intègre une surveillance WebSocket en temps réel et applique une politique stricte de *non-repli silencieux* : plutôt que de dégrader discrètement la qualité, le système échoue de manière explicite.

Description courte

Outil universel de traduction d'ebooks basé sur Go. Prend en charge les formats FB2, EPUB, TXT, HTML, PDF et DOCX pour plus de 100 langues, en utilisant les meilleurs modèles LLM vérifiés (via le pont LLMsVerifier). Intègre des API REST/HTTP-3 et gRPC, un traitement distribué et un tableau de bord de surveillance WebSocket en temps réel.

Description longue

HelixTranslate est un système d'entreprise, basé sur Go, conçu pour traduire des livres entiers d'une langue à l'autre en s'appuyant sur des fournisseurs LLM — non pas des paragraphes ou des extraits, mais des œuvres complètes, de bout en bout. Il analyse et régénère plusieurs formats d'ebooks (FB2, EPUB, TXT, HTML, PDF, DOCX), prend en charge plus de 100 langues avec détection automatique, et propose à la fois des outils CLI et des serveurs API (REST sur HTTP/3, gRPC, ainsi qu'un flux d'événements WebSocket), ce qui lui permet de s'intégrer aussi bien à un workflow en terminal qu'à un maillage de services. Sa

caractéristique déterminante réside dans *la manière dont il choisit un modèle* : au lieu de coder en dur un fournisseur en espérant qu’il reste opérationnel, HelixTranslate délègue toute autorité sur les modèles au pont LLMsVerifier (pkg/bridge), qui sélectionne le modèle API *vérifié* le plus performant et renvoie une chaîne de repli déterministe, classée par score. L’éligibilité d’un modèle est déterminée par un score pondéré prenant en compte la réactivité, le code, la richesse fonctionnelle et la fiabilité — ainsi, le modèle chargé de votre traduction a gagné sa place en prouvant son efficacité, et non en figurant simplement dans un fichier de configuration.

De manière cruciale, le système applique un principe de « *non-repli silencieux* » directement intégré au code : si aucune clé de fournisseur API n’est présente, ou si un opérateur demande explicitement un fournisseur indisponible, le pipeline renvoie une erreur franche plutôt que de basculer discrètement vers un autre fournisseur ou de recourir à un moteur local en prétendant que tout va bien — une règle renforcée par une vérification pré-compilation dédiée et un test de mutation associé. Les moteurs locaux (Ollama, llama.cpp) ont été délibérément retirés du chemin par défaut afin qu’un moteur moins performant ne puisse jamais remplacer silencieusement un modèle vérifié. Autour du cœur de traduction s’articule un sous-système de surveillance WebSocket en temps réel : l’outil de traduction CLI émet des événements typés vers un serveur de monitoring qui alimente un tableau de bord web en direct, tandis que des workers SSH distants répartissent la charge pour une traduction distribuée. S’y ajoutent un polissage multi-passes pour la cohérence, une analyse de qualité en phase de préparation, un cache de traduction pour maîtriser les coûts sur les longs textes, et un contrôle qualité visuel. L’ensemble de la plateforme répond à une constitution d’ingénierie anti-bidon : les tests doivent prouver des résultats concrets et visibles par l’utilisateur, soutenus par des tests de mutation obligatoires plutôt que par des coches vertes qui ne prouvent rien.

Contenu

Pourquoi nous l’avons conçu

Traduire des ouvrages longs de manière fiable et *honnête* — sans jamais livrer une traduction « dégradée mais présente ». Notre principe de conception repose sur une idée simple : une traduction manquante ou invérifiable doit être considérée comme une erreur manifeste et bruyante, et le choix du modèle doit toujours aboutir à un fournisseur authentiquement vérifié, plutôt qu’à une supposition codée en dur ou à un repli local silencieux.

Pourquoi c'est révolutionnaire

La plupart des pipelines de traduction HelixTranslate échouent en silence — ils basculent discrètement vers un modèle moins performant, se rabattent sur un runtime local ou produisent un résultat partiel, tandis que la suite de tests reste verte et que personne ne remarque la chute brutale de qualité. HelixTranslate rend ce mode de défaillance structurellement impossible : le choix du modèle est soumis à une vérification, la chaîne de repli est déterministe et entièrement transparente, et l'absence de clé ou de modèle vérifié se solde par une erreur franche plutôt qu'un haussement d'épaules silencieux. Cette seule décision de conception transforme la question « *Cette traduction a-t-elle vraiment été exécutée par un modèle compétent et vérifié ?* » d'un espoir invérifiable en une garantie que le système impose en votre nom.

Ce qui est innovant

- **Routage des modèles contrôlé par vérification** via le pont LLMsVerifier — le modèle *vérifié* le plus performant est sélectionné automatiquement, permettant aux opérateurs d'exprimer une intention plutôt que des noms de fournisseurs, et évitant de choisir manuellement un prestataire susceptible d'être indisponible.
- **Garantie de non-repli silencieux imposée par le code** — quatre branches de routage explicites (simulation / vérificateur explicite / fournisseur explicite / pont par défaut), chacune générant une erreur franche plutôt qu'un basculement silencieux, complétées par la suppression délibérée des runtimes locaux du chemin par défaut, de sorte qu'il n'existe aucun niveau inférieur vers lequel replier.
- **Application mécanique** — une porte de pré-construction CM-NO-LOCAL-RUNTIME associée à un test de mutation vérifié, au moment de la compilation, qu'aucun client de runtime local n'est jamais instancié sur le chemin par défaut : la garantie ne peut se dégrader, car la compilation échoue si c'est le cas.
- **Chaîne de repli déterministe et classée par score** — le basculement entre fournisseurs de modèles *vérifiés* est autorisé et entièrement transparent, une distinction de principe par rapport au repli silencieux interdit : vous savez toujours quel modèle compétent a pris en charge le travail.
- **Surveillance WebSocket en temps réel** — des événements de traduction typés diffusés en direct sur un tableau de bord, avec des travailleurs SSH distribués, rendant un projet de traduction d'un ouvrage visible et parallèle, et non une boîte noire.
- **Régime de tests anti-bidon** — tests de mutation, assertions négatives, exécutions sur système réel et contrôle qualité visuel garantissent ensemble que « *les tests passent* » ne peut

jamais masquer discrètement « *la fonctionnalité ne fonctionne pas réellement* ».

Principaux défis techniques et nos solutions

- **Garantir un pipeline de traduction honnête (sans dégradation silencieuse).** Résolu en centralisant toute l'autorité des modèles dans le pont LLMsVerifier, créant ainsi un point de décision unique à contrôler, en codant quatre branches de routage explicites qui échouent bruyamment plutôt que de deviner, en supprimant entièrement les replis vers des runtimes locaux du chemin par défaut, et en verrouillant la règle avec une porte de compilation et un test de mutation qui fait échouer la compilation si la garantie est un jour retirée.
- **« Tests verts, fonctionnalités cassées. »** La constitution nomme explicitement ce mode de défaillance et le combat grâce au régime de tests anti-bidon : des assertions concrètes et visibles par l'utilisateur plutôt que des détails d'implémentation, des systèmes réels intégrés (les simulations étant cantonnées aux tests unitaires), des tests de mutation obligatoires (casser délibérément la fonctionnalité doit faire échouer le test), et un contrôle qualité visuel qui examine réellement le résultat.
- **Qualité sur des formats longs et variés.** Les entrées de la longueur d'un livre mettent à rude épreuve la cohérence et le budget ; solution apportée par un polissage en plusieurs passes qui réexamine le texte, une phase de préparation analysant la charge de travail en amont, et un cache de traduction évitant de payer deux fois pour le même passage.

Contenu

Pile technologique

- **Go** — choisi pour ses primitives de concurrence, qui s'adaptent naturellement au parsing, à la traduction et au streaming simultanés de nombreux chapitres ; le backend à haute concurrence, module `digital.vasic.translator`.
- **Gin** — retenu comme routeur HTTP rapide et minimaliste pour servir la surface REST API.
- **QUIC / HTTP/3 (quic-go)** — adopté pour doter la surface REST API d'un transport moderne à faible latence, résistant aux réseaux imparfaits.
- **gRPC + Protocol Buffers** — sélectionné pour une interface de service fortement typée et performante, fonctionnant en parallèle de REST pour les appels programmatiques.

- **Gorilla WebSocket** — utilisé pour acheminer le flux d'événements de traduction typés en temps réel, alimentant en direct le tableau de bord de supervision.
- **PostgreSQL, SQLite, Redis** — une répartition délibérée en trois niveaux : PostgreSQL pour les données relationnelles durables, SQLite pour l'état local/embarqué (il sert également de support à la base de modèles vérifiés, `data/verified_models.db`), et Redis comme cache actif.
- **unidoc/unioffice + unipdf** — retenus pour gérer les formats complexes : le parsing et la régénération des fichiers DOCX et PDF, afin que les livres numériques multi-formats effectuent un aller-retour fidèle.
- **Cobra** — choisi comme framework CLI alimentant `unified-translator` et ses outils associés.
- **golang-jwt (JWT HS256)** — adopté pour l'authentification API sans état, couplé à une limitation de débit par jeton et par IP, ainsi qu'à la sécurité des transports TLS/QUIC pour renforcer la surface.
- **Pont LLMsVerifier (pkg/bridge)** — pièce maîtresse : il fournit le modèle vérifié le plus robuste ainsi que sa chaîne de repli déterministe, et sert de point de contrôle unique pour garantir l'absence de repli silencieux.
- **Testify** — retenu pour la suite de tests Go, incluant le fichier dédié `provider_routing_test.go` et les portes de mutation qui préservent l'intégrité des règles d'honnêteté.
- **Docker / Podman (sans root) + Compose** — choisis pour un déploiement conteneurisé et distribué (`docker-compose.distributed.yml`), avec Podman sans root pour un profil de sécurité plus strict.

Statut et notes d'honnêteté

- **Statut : bêta.** Plateforme fonctionnelle ; la version est indiquée de manière incohérente entre `VERSION/Makefile/AGENTS.md`, et est donc considérée comme non stabilisée.
- **Licence : à déterminer.** Le README mentionne la licence MIT, mais cela n'a pas été confirmé par un fichier `LICENSE` — à vérifier avant toute affirmation.
- Les points de terminaison du tableau de bord/monitor sont réservés à localhost et ne sont pas publics. Les performances WebSocket indiquées dans la documentation sont des objectifs, non des valeurs vérifiées. Le fichier `ARCHITECTURE.md` répertorie encore des moteurs Ollama/locaux supprimés (obsolète).

Niveau de priorité : Helix-primaire (grappe LLM-infrastructure).
Classé au sein de la famille de plateformes Helix, après HelixTrack.